

Bellman-Ford Algorithm

Handling Negative Weights and Cycle Detection

1. What is the Bellman-Ford Algorithm?

While Dijkstra's is faster, the **Bellman-Ford Algorithm** is more versatile. It finds the shortest path from a single source to all other vertices in a directed graph, even when **negative edge weights** are present.

Primary Function: Bellman-Ford is the only standard shortest-path algorithm capable of detecting **Negative Cycles**, a condition where the total weight of a loop is less than zero.

2. Working Mechanism: Iterative Relaxation

The algorithm iterates through all edges exactly **V-1** times (where V is the number of vertices). This ensures that the shortest path information propagates throughout the graph.

- **Initialization:** Distance to source = 0, all others = Infinity.
- **The Main Loop:** For (V-1) iterations, relax every edge (u, v) in the graph.
- **Result:** After V-1 passes, the distance array contains the actual shortest paths.

3. Detecting Negative Cycles

Since a shortest path in a graph without negative cycles can have at most $V-1$ edges, any further relaxation possible after the $(V-1)$ -th pass indicates the presence of a negative cycle.

```
# Detecting after V-1 passes
for u in range(self.size):
    for v in range(self.size):
        if distance[u] + weight(u, v) < distance[v]:
            return "Negative Cycle Detected!"
```

4. Performance and Complexity

The time complexity of Bellman-Ford is higher than Dijkstra's, making it less suitable for massive, positive-only graphs:

Time Complexity: $O(V * E)$
Space Complexity: $O(V)$

In a fully connected (dense) graph where $E \approx V^2$, the complexity effectively becomes $O(V^3)$.

5. Python Implementation

This snippet demonstrates the core relaxation logic within the source graph:

```
for i in range(self.size - 1):
    for u in range(self.size):
        for v in range(self.size):
            if self.adj_matrix[u][v] != 0:
                new_dist = distances[u] + self.adj_matrix[u][v]
```

```
if new_dist < distances[v]:  
    distances[v] = new_dist
```

CodeHive

Comparison with Dijkstra (Part 1)

The fundamental trade-off between the two algorithms is **Flexibility vs. Speed**. Dijkstra's is a greedy algorithm that assumes the local optimum is part of the global optimum—this fails when a negative weight might make a previously 'longer' path cheaper later on. Bellman-Ford avoids this by checking all possibilities systematically.

Practical Application: In distance-vector routing protocols (like RIP), routers use a variation of Bellman-Ford to find the best paths through a changing network.

Visualizing Distance Propagation

In the first iteration, only nodes directly connected to the source get updated. In the second iteration, nodes connected to those nodes get updated. By the $(V-1)$ -th iteration, the furthest possible node from the source (in terms of edge count) has been reached.

Comparison with Dijkstra (Part 2)

The fundamental trade-off between the two algorithms is **Flexibility vs. Speed**. Dijkstra's is a greedy algorithm that assumes the local optimum is part of the global optimum—this fails when a negative weight might make a previously 'longer' path cheaper later on. Bellman-Ford avoids this by checking all possibilities systematically.

Practical Application: In distance-vector routing protocols (like RIP), routers use a variation of Bellman-Ford to find the best paths through a changing network.

Visualizing Distance Propagation

In the first iteration, only nodes directly connected to the source get updated. In the second iteration, nodes connected to those nodes get updated. By the $(V-1)$ -th iteration, the furthest possible node from the source (in terms of edge count) has been reached.